

Retrieval Engine Competition Project

Project Report

Group: SeaFour

Maksym DOLHOV

Mehdi AGHAEI

Nguyen Ho Bao KHANH

Submission scope. This final version brings together the saved offline evaluation notebook `retrieval-evaluation.ipynb`, the submission notebook `kaggle-submission.ipynb`, and the public Kaggle score. It covers dataset exploration, preprocessing, embedding inspection, a Phase 1 comparison on all 327 labeled queries, document-holdout classification, Phase 2 retrieval improvements for all three first-stage models, and the regenerated `solutions_SeaFour.csv` file.

Main notebook	<code>kaggle-submission.ipynb</code>
Main output	<code>solutions_SeaFour.csv</code>
Public Kaggle score	0.62419
Date	April 5, 2026

1 Introduction

Information retrieval is the task of ranking documents so that the most relevant ones appear first for a user query. In this project we work with a large technical corpus and a set of labeled training queries. The assignment is divided into two phases. Phase 1 focuses on basic retrieval with classical and neural methods. Phase 2 adds category classification and asks how the classifier can be used to improve retrieval.

- dataset loading and statistical exploration;
- construction of a normalized `content` field;
- three retrieval models: TF-IDF, BM25+, and Sentence-Transformer embeddings;
- embedding inspection through matrix shapes and a 2D projection;
- evaluation with Recall@K, Precision@K, and MRR@K;
- category classification for both documents and queries;
- classifier-aware reranking for Phase 2;
- Kaggle submission export.

2 Project Roadmap

This report is not only about the final score, but also about how the system improved over time. We started with lexical retrieval, moved to semantic retrieval, added category prediction as an extra signal, and then turned everything into a submission-ready pipeline after a few important debugging and reproducibility fixes.

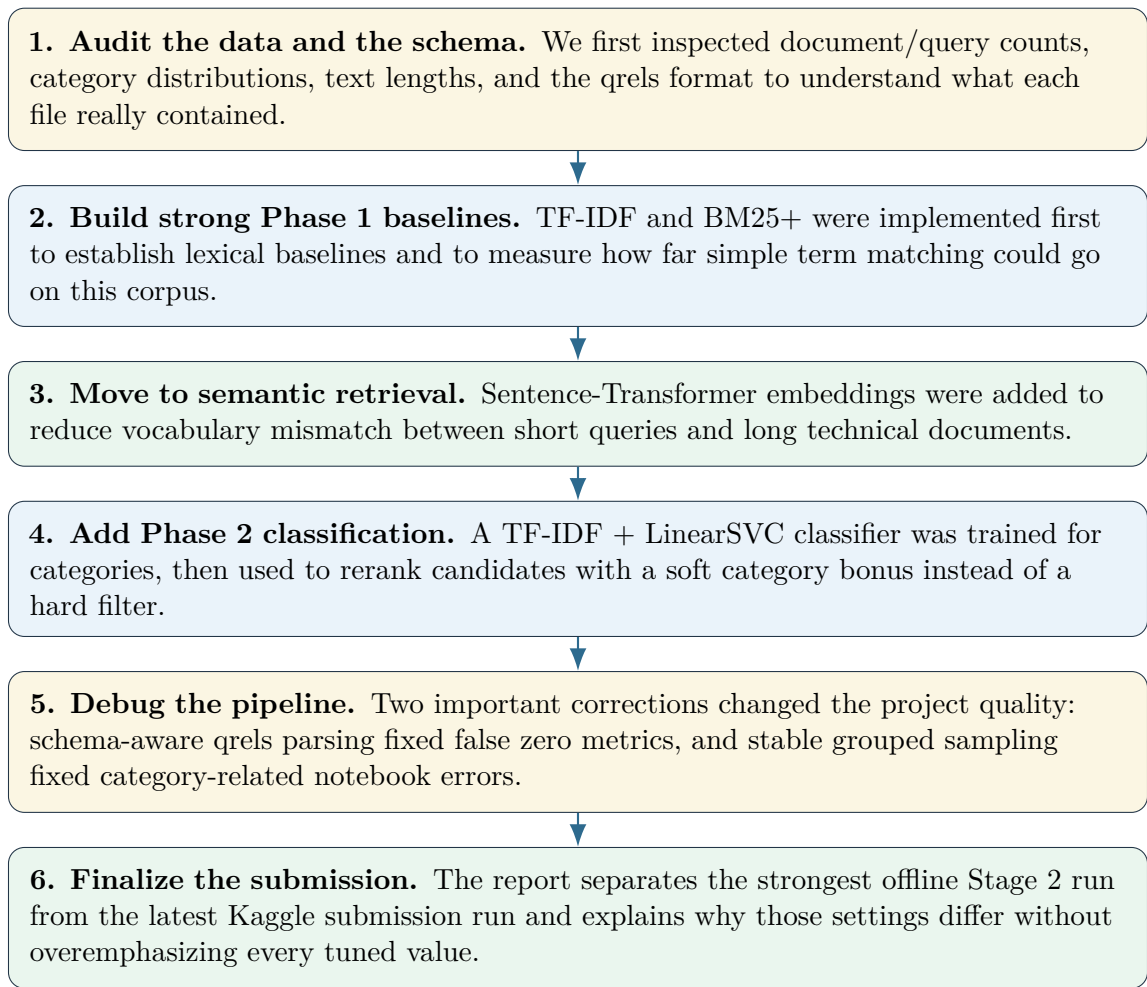


Figure 1: Roadmap from the first baselines to the final submission-ready system.

Stage	Main question	Outcome
Data audit	What is in the files, and how should they be interpreted?	We clarified the schema, the category structure, the query/document asymmetry, and the relevance format used by the qrels.
Lexical baselines	How far can exact-term retrieval go?	TF-IDF and BM25+ gave interpretable baselines, but they remained limited by lexical mismatch.
Dense retrieval	Can semantic matching improve recall and rank quality?	Embeddings became the strongest retriever by a large margin on Recall and MRR.
Classification	Can category prediction help retrieval?	Yes, but as a soft reranking signal rather than as a strict filter.
Debugging	Why did some notebook outputs look suspicious or fail?	Fixing qrels parsing and grouped sampling removed false zeros and notebook instability.
Submission	Which pipeline felt solid enough to submit?	Embeddings stayed at the core, while later runs added a tag-aware query classifier, a soft category bonus, and cross-encoder reranking. The exact settings differ between the saved evaluation run and the current code defaults.

Table 1: Milestones that structured the project from start to final submission.

3 Final Changes and Reasons

The final version differs from our earlier drafts in five main ways.

1. **Ground-truth parsing now matches the real qrels schema.** The competition qrels store relevant items under `relevant_doc_ids` with nested `doc_id` entries. Without this fix, Recall, Precision, and MRR could be computed against empty relevance sets. The notebook now also raises an error if qrels parsing produces empty relevance, misses labeled queries, or references document ids that are not in the corpus.
2. **Category-wise sampling is now stable with the current pandas behavior.** The notebook uses a sampling pattern that keeps the `category` column during visualization and downsampling. This avoids the grouped-sampling issue that was breaking category-aware plots and classifier preparation.
3. **The report now separates labeled-query evaluation from Kaggle submission runs.** Only the training queries have visible qrels, while the Kaggle test queries are unlabeled. Because of that, the report keeps offline Stage 2 analysis separate from the later submission run.
4. **The discussion now focuses more on the main behaviors than on the full hyperparameter history.** Earlier drafts put too much emphasis on one supposedly “current” Stage 2 configuration. The revised version focuses instead on the patterns that stayed consistent across later experiments: strong embedding recall, useful reranking, harmful hard filtering, and small gains from a soft category bonus.
5. **The final report keeps three pieces of evidence separate.** The best offline Stage 2 result comes from `retrieval-evaluation.ipynb`; the current submission setup comes from `src/config.py` together with `kaggle-submission.ipynb`; and the public score 0.62419 comes from the leaderboard.

3.1 Previous tries, failures, and lessons

Baseline retrieval attempts.

- **TF-IDF baseline.** The model was simple and useful as a first sanity check, but lexical overlap alone was not enough for many relevant query-document pairs. In the end, TF-IDF stayed useful as a baseline, not as our final retriever.
- **BM25+ lexical upgrade.** BM25+ improved over TF-IDF and handled document-length variation better, but it still stayed clearly behind embeddings on recall and rank quality. We kept it as our strongest lexical reference point rather than our final Stage 1 model.

Notebook and pipeline failures.

- **Grouped sampling instability.** A grouped pandas sampling pattern sometimes dropped the `category` column and triggered notebook errors such as `KeyError: category`. The fix was to switch to stable category-preserving downsampling for visualization and classifier preparation.

Lessons that shaped the final report.

- **Query classifier with text + tags.** The classifier became extremely accurate because the query tags are almost category-pure in the labeled split, so they behave a lot like label-revealing metadata. For that reason, we keep this caveat visible and discuss the classifier mainly through its effect on retrieval decisions rather than presenting the raw setup as a headline result.
- **Evaluation depth vs. submission depth.** Later notebooks and the current code defaults do not all use exactly the same depth values. So the offline evaluation, the current submission defaults, and the public Kaggle result need to be reported separately. What matters most in practice is how depth changes coverage, reranking reach, runtime, and score interpretation.

4 Dataset Description and Exploratory Analysis

In `data-analysis.ipynb`, we found **216,041 documents**, **327 labeled training queries**, **141 Kaggle test queries**, and **2,970 relevance judgments**. The documents belong to **five categories**. Since only `queries_train.json` has visible qrels, we use all 327 labeled queries for offline evaluation, while the 141 Kaggle test queries are kept for the final submission export. Tables 2–4 summarize the dataset size, the role of each query file, and the category distribution.

Dataset element	Count or value
Documents	216,041
Labeled training queries	327
Kaggle test queries	141
Relevance judgments	2,970
Categories	5
Average relevant documents per training query	9.08
Average document length	900.92 characters / 155.81 tokens
Average training query length	48.44 characters / 8.55 tokens
Average test query length	48.88 characters / 8.50 tokens

Table 2: Global dataset overview.

Notebook role	Number of queries
Labeled train queries used for offline diagnostics	327
Kaggle test queries used for final export	141

Table 3: How the report uses the available query files.

Category	Document count	Training-query count
<code>tex</code>	68,184	130
<code>unix</code>	47,382	47
<code>gaming</code>	45,301	41
<code>programmers</code>	32,176	52
<code>android</code>	22,998	57

Table 4: Document and labeled-query distribution by category.

Figure 2 gives a more visual view of the same category distribution. It shows that `tex` is the largest category in the corpus, while `android` is the smallest. The categories are not perfectly balanced, but every class is still large enough to support both retrieval and classification experiments.

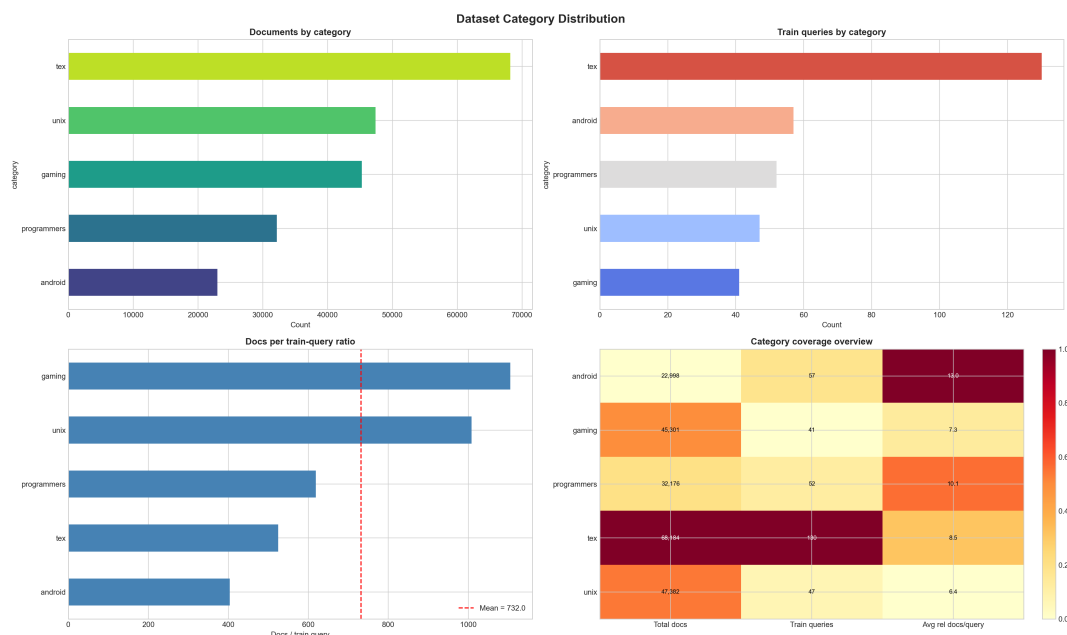


Figure 2: Document distribution by category produced by `data-analysis.ipynb`.

These exploratory results matter because they help explain later retrieval behavior. In particular, the corpus contains long documents while the queries are much shorter. This asymmetry generally makes strict lexical overlap less reliable, especially when the query paraphrases the document instead of reusing its exact surface vocabulary.

4.1 Available fields

The notebook lists the following fields:

- documents: `id`, `title`, `text`, `tags`, `category`;
- training queries: `id`, `title`, `text`, `tags`, `category`;
- test queries: `id`, `title`, `text`, `tags`;
- qrels: `category`, `relevant_doc_ids`, `total_relevant_docs`, with nested `doc_id` and `relevance`.

The qrels loader validates that every labeled query has at least one relevant document and that the referenced document ids exist in the corpus.

5 Preprocessing and Content Construction

The assignment asks for a new text field called `content`. The notebook uses the following policy:

- documents: `title + text + tags`;
- retrieval queries: `title + text`;
- classifier queries in the active notebook: `title + text + tags`.

So the project uses different query views for retrieval and for Stage 2 classification. Retrieval stays close to the original query wording through `title + text`, while the category classifier also uses `tags`. This helped the practical submission pipeline, but it has to be interpreted carefully because the query tags are highly informative about category in the labeled data.

The shared normalization steps are:

- lowercasing;

- replacement of the separators -, _, and / with spaces;
- whitespace collapsing;
- trimming leading and trailing whitespace.

For lexical methods, tokenization uses the pattern `[a-z0-9]+`, and English stopwords filtering is enabled by default.

6 Phase 1: Retrieval Models

6.1 TF-IDF retrieval

TF-IDF stands for *term frequency-inverse document frequency*. It gives high weight to terms that appear frequently in one document but are relatively rare in the collection. In vector-space retrieval, each document and query is represented by a sparse TF-IDF vector, and documents are ranked by cosine similarity with the query vector.

Conceptually, TF-IDF is useful because it is simple, fast, and often surprisingly competitive as a baseline. Its main limitation is that it depends strongly on lexical overlap. If a relevant document uses different wording from the query, TF-IDF may fail even when the underlying meaning is similar.

6.2 BM25+ retrieval

BM25+ is a probabilistic lexical ranking model. It improves over naive bag-of-words weighting by:

- saturating term frequency contributions;
- normalizing for document length;
- adding a positive lower-bound adjustment through the BM25+ variant.

BM25+ is often stronger than plain TF-IDF when document lengths vary substantially, as is the case in this corpus. However, BM25+ remains a lexical model and therefore still depends on token overlap.

6.3 Embedding retrieval

Dense embeddings map text into high-dimensional vectors that encode semantic information. In this project we use the Sentence-Transformer model `all-MiniLM-L6-v2`. Documents and queries are embedded into a 384-dimensional space, and cosine similarity is computed through normalized dot products.

Embeddings can help retrieval because they are more robust to synonyms, paraphrases, and broader semantic similarity. Their main tradeoffs are higher computational cost, larger memory usage, and the risk of retrieving semantically related but not strictly relevant documents.

7 Embedding Inspection and Visualization

The notebook prints the exact embedding matrix shapes:

- document embeddings: (216,041, 384);
- labeled-train-query embeddings: (327, 384).

This is important because the assignment asks us to inspect the embedding structures directly, not just mention them in words.

To make the geometry more interpretable, the notebook also builds a 2D projection using **t-SNE** with `random_state=42`. The plot uses a category-balanced document sample together with all labeled

training queries. The resulting figure shows partial clustering by category, which is expected because **android**, **gaming**, **tex**, **unix**, and **programmers** contain distinctive technical vocabulary and recurrent semantic themes.

At the same time, perfect separation should not be expected. The original embeddings live in a much higher-dimensional space, some categories overlap semantically, and short queries are often more ambiguous than documents.

Figure 3 shows the 2D embedding projection produced by the notebook. The visible grouping confirms that the dense representation captures meaningful category structure, while the overlaps also show that the classes are not trivially separable in two dimensions.

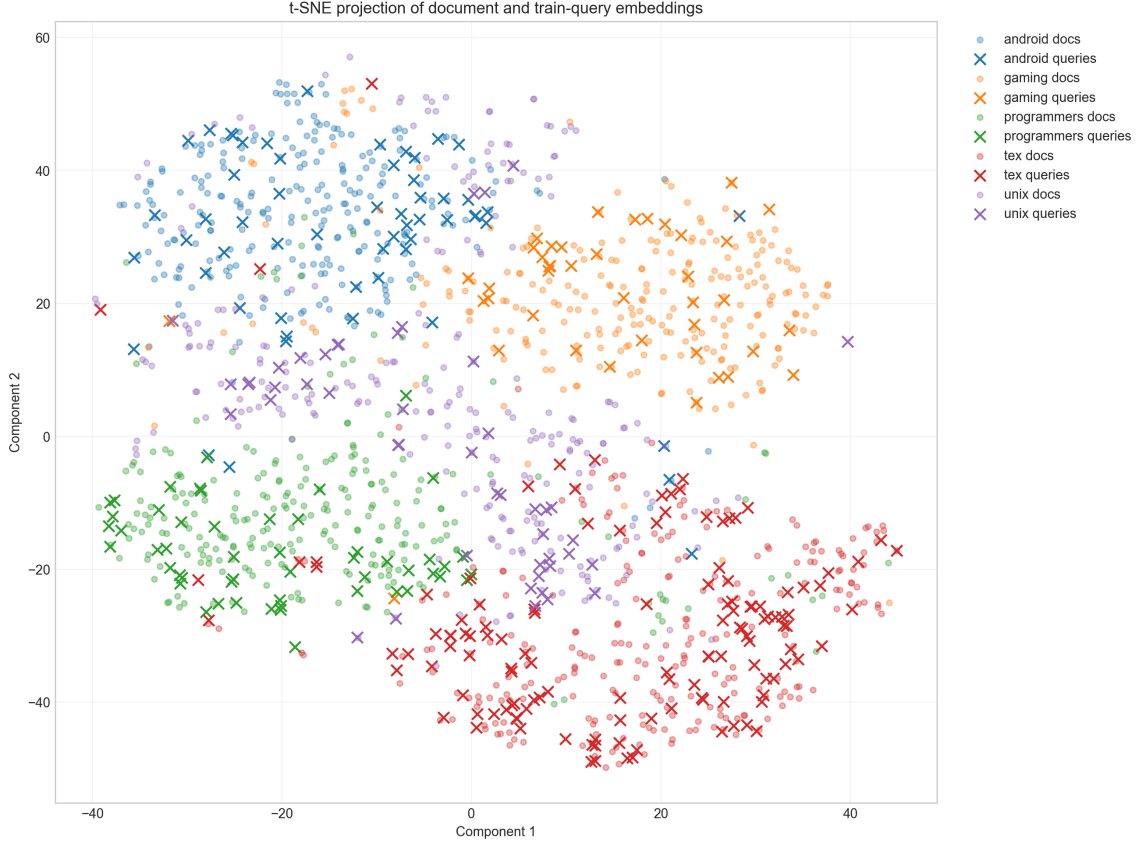


Figure 3: 2D t-SNE projection of sampled document embeddings and all labeled training-query embeddings produced by `data-analysis.ipynb`.

8 Evaluation Measures

The notebook implements the three required retrieval metrics:

$$\text{Recall@K} = \frac{\text{\#relevant retrieved documents}}{\text{\#relevant documents}}$$

$$\text{Precision@K} = \frac{\text{\#relevant retrieved documents}}{\text{\#retrieved documents}}$$

$$\text{MRR@K} = \frac{1}{\text{rank of the first relevant document}}$$

For Phase 1 comparison, the notebook also reports runtime and average latency per query. For Phase 2, it augments the retrieval summary with classifier accuracy, and computes a combined score as

$$\text{CombinedScore} = \frac{1}{4} (\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy}).$$

9 Phase 1 Experimental Protocol

Only `queries_train.json` has relevance labels, while `queries_test.json` is the unlabeled competition test set. So in the final notebook run we use **all 327 labeled training queries** as the offline evaluation set for retrieval. We do not introduce an additional query-side validation split.

Phase 1 compares TF-IDF, BM25+, and embeddings over the grid

$$K \in \{10, 100, 1000, 7500, 12500\}.$$

Each model is run once at $K = 12500$, and the smaller- K metrics are obtained by truncating the stored ranked lists. For each model the notebook stores:

- top-k document ids;
- top-k retrieval scores;
- Recall@K, Precision@K, and MRR@K;
- runtime and latency.

The explicit arrays requested by the assignment are preserved in the notebook as:

- `topk_indices_tfidf`, `topk_scores_tfidf`;
- `topk_indices_bm25`, `topk_scores_bm25`;
- `topk_indices_embedding`, `topk_scores_embedding`.

In the saved notebook run, all six arrays have shape $(327, 12500)$, which answers that assignment point directly.

9.1 Phase 1 results and effect of K

Table 5 reports the full Phase 1 comparison extracted from `retrieval-evaluation.ipynb`.

Model	K	Recall	Precision	MRR	Combined score
BM25+	10	0.151892	0.092355	0.271822	0.360668
BM25+	100	0.318234	0.020795	0.281283	0.386729
BM25+	1000	0.510815	0.003734	0.281970	0.430781
BM25+	7500	0.681980	0.000702	0.282020	0.472827
BM25+	12500	0.717214	0.000449	0.282023	0.481573
Embedding	10	0.311335	0.183486	0.466540	0.471992
Embedding	100	0.581491	0.040489	0.474049	0.505659
Embedding	1000	0.828389	0.006431	0.474495	0.558980
Embedding	7500	0.947617	0.001099	0.474507	0.587457
Embedding	12500	0.965746	0.000685	0.474507	0.591886
TF-IDF	10	0.140535	0.085933	0.251524	0.351149
TF-IDF	100	0.324239	0.021284	0.261963	0.383523
TF-IDF	1000	0.559925	0.004089	0.262662	0.438320
TF-IDF	7500	0.765464	0.000780	0.262707	0.488889
TF-IDF	12500	0.790170	0.000490	0.262707	0.494993

Table 5: Phase 1 comparison across all models and all evaluated K values. The combined score is the notebook’s average of Recall, Precision, MRR, and classifier accuracy.

The pattern across K is quite clear in Table 5. As K increases, Recall goes up for all three models because more relevant documents can enter the candidate list. Precision goes down because the list becomes much longer than the number of extra relevant hits. MRR changes only a little after the first few ranks, which means that larger K mainly improves *coverage*, not the position of the first relevant result.

Model	Best K	Recall@K	Precision@K	MRR@K	Latency/query
TF-IDF	12500	0.790170	0.000490	0.262707	365.34 ms
BM25+	12500	0.717214	0.000449	0.282023	437.68 ms
Embeddings	12500	0.965746	0.000685	0.474507	3.68 ms

Table 6: Best Phase 1 result per model, including measured latency.

The Phase 1 results suggest four main points.

- **Embeddings were the strongest retriever at every evaluated K .** They achieved the highest Recall and the highest combined score over the full grid.
- **TF-IDF ended up with higher recall than BM25+ at large K , while BM25+ kept the better MRR.** In other words, TF-IDF retrieved more relevant material by depth 12500, but BM25+ ranked the first relevant result slightly earlier on average.
- **All three models reached their best combined score at $K = 12500$ in the saved notebook run.** The gain mainly came from higher Recall; MRR was already close to saturated by $K = 1000$.
- **The embedding implementation was also the fastest.** With cached document embeddings and chunked query scoring, the dense retriever was much faster than the current TF-IDF and BM25+ loops in local execution.

System stage	Recall@K	Precision@K	MRR@K	Interpretation
TF-IDF baseline	0.790170	0.000490	0.262707	Stronger long-list recall than BM25+, but still limited by lexical mismatch and weaker early ranking.
BM25+ lexical model	0.717214	0.000449	0.282023	The stronger lexical ranker near the top of the list, but still clearly behind embeddings on coverage.
Embedding retrieval	0.965746	0.000685	0.474507	The main Phase 1 jump: semantic matching dominates both lexical baselines and sets up a stronger Phase 2 reranking stage.

Table 7: How the retrieval project progressed during Phase 1.

10 Phase 2: Category Classification

10.1 Classifier model and input features

In the notebook, we use a **TF-IDF + LinearSVC** classifier. This is a reasonable choice for a five-class text classification problem, and it is simple enough to train and evaluate quickly. The input features are:

- document classifier text: `title + text + tags`;
- query classifier text for diagnostics and submission-time hints: `title + text + tags`.

For the clean standalone classification benchmark, the notebook trains on an **80/20 stratified document split** with `random_state=42`. Queries are *not* mixed into that training split. They are only used later as a diagnostic transfer test.

10.2 How accuracy is computed

The notebook computes accuracy as

$$\text{Accuracy} = \frac{\text{\#correct category predictions}}{\text{\#examples in the evaluation set}}.$$

This definition is used both for the held-out document benchmark and for the diagnostic query-side comparison.

10.3 Document holdout results

The held-out document benchmark is very strong: the notebook reports **document-holdout accuracy** = 0.986045. Table 8 shows the per-category precision, recall, F_1 , and per-category accuracy taken from `retrieval-evaluation.ipynb`.

Category	Precision	Recall	F_1	Accuracy
android	0.985291	0.975652	0.980448	0.975652
gaming	0.993385	0.994481	0.993933	0.994481
programmers	0.977089	0.974204	0.975644	0.974204
tex	0.994060	0.993987	0.994023	0.993987
unix	0.973982	0.979635	0.976800	0.979635

Table 8: Held-out document classification results from the stratified 80/20 document split.

Performance is not exactly the same across all categories. In this run, **gaming** and **tex** are the easiest classes, while **android** and **programmers** are slightly harder. Even so, the weakest category still stays above 0.97 per-category accuracy.

10.4 Document-vs-query comparison

The assignment also asks whether the classifier works equally well on documents and queries. To answer that, the notebook scores the document-trained classifier on the 327 labeled training queries:

Evaluation set	Accuracy	Samples
Document holdout	0.986045	43,209
Labeled train queries (diagnostic)	0.932722	327

Table 9: Document-vs-query category classification comparison.

So the answer is **no**: the classifier does *not* work equally well on documents and queries. It transfers well to queries, but the diagnostic query accuracy is lower than the held-out document accuracy, which is consistent with the fact that queries are much shorter and more ambiguous than full documents.

11 Using the Classifier to Improve Retrieval

The classifier is integrated into retrieval in two ways:

- **hard category filtering**, which restricts retrieval to documents from the predicted category;
- **soft category-aware reranking**, which keeps the candidate list and adds a category-match bonus during cross-encoder reranking.

For this part of the report, we keep three sources of evidence separate:

- **saved offline evaluation outputs** from `retrieval-evaluation.ipynb`;
- **the current submission configuration** from `src/config.py` and `kaggle-submission.ipynb`;
- the **public Kaggle score** 0.62419.

We keep them separate so that we do not pretend everything came from one identical run. The saved offline tables are the source of the Phase 2 metrics below. The submission notebook shows the current export pipeline and the regenerated CSV preview. The public Kaggle score 0.62419 is an external leaderboard result and cannot be reproduced from the local notebooks alone.

Run context	Relevant settings tied to the result	Impact on interpretation
Saved offline evaluation run	All 327 labeled queries, $K = 12500$, <code>rerank_top_m=45</code> , <code>category_bonus=0.5</code> , TF-IDF/BM25+/embedding all compared	Measures how Stage 2 choices affect Recall, MRR, and combined score on labeled queries. This is the source of the Phase 2 comparison tables.
Current submission notebook	<code>final_model=embedding</code> , <code>submit_top_k=12500</code> , <code>rerank_top_m=45</code> , <code>category_bonus=0.5</code> , query tags enabled for classification	Describes how the code now exports and reranks <code>solutions_SeaFour.csv</code> . These settings explain the current pipeline shape, but they are not themselves an offline benchmark.
External Kaggle result	Public leaderboard score = 0.62419	Confirms that the selected Stage 2 direction transfers to the hidden test set, but should not be read as a literal replay of the saved offline evaluation run.

Table 10: Offline outputs, the current submission configuration, and the public Kaggle result are reported as separate sources of evidence.

11.1 Offline Stage 2 results and what they mean

The saved outputs in `retrieval-evaluation.ipynb` provide the clearest offline comparison of the main Stage 2 variants. Table 11 reports all three first-stage models and all five classifier-aware variants at $K = 12500$.

Model	Variant	Recall	Precision	MRR	Combined score	Δ vs. baseline
TF-IDF	baseline	0.790170	0.000490	0.262707	0.494993	0.000000
TF-IDF	category filtered	0.795661	0.000535	0.253238	0.494010	-0.000983
TF-IDF	reranked	0.790170	0.000490	0.478497	0.548940	+0.053947
TF-IDF	reranked + category bonus	0.790170	0.000490	0.480366	0.549408	+0.054415
TF-IDF	category filtered + rerank	0.795661	0.000535	0.472139	0.548735	+0.053742
BM25+	baseline	0.717214	0.000449	0.282023	0.481573	0.000000
BM25+	category filtered	0.737898	0.000502	0.269339	0.483586	+0.002013
BM25+	reranked	0.717214	0.000449	0.467999	0.528067	+0.046494
BM25+	reranked + category bonus	0.717214	0.000449	0.468184	0.528113	+0.046540
BM25+	category filtered + rerank	0.737898	0.000502	0.429715	0.523680	+0.042107
Embedding	baseline	0.965746	0.000685	0.474507	0.591886	0.000000
Embedding	category filtered	0.905954	0.000657	0.442439	0.568914	-0.022972
Embedding	reranked	0.965746	0.000685	0.643416	0.634113	+0.042227
Embedding	reranked + category bonus	0.965746	0.000685	0.648262	0.635325	+0.043439
Embedding	category filtered + rerank	0.905954	0.000657	0.601620	0.608709	+0.016823

Table 11: Offline Stage 2 comparison from `retrieval-evaluation.ipynb` at $K = 12500$.

These results point to five main conclusions.

1. **The best overall offline result is the embedding retriever with reranking and a soft category bonus.** It reaches a combined score of **0.635325** with $\text{MRR} = 0.648262$.
2. **Reranking is the main source of Phase 2 gains for every first-stage model.** Relative to each baseline, reranked + bonus improves the combined score by +0.054415 for TF-IDF, +0.046540 for BM25+, and +0.043439 for embeddings.
3. **The soft category bonus helps, but only slightly.** For all three first-stage models, reranked + bonus is a little better than reranked-only, which means the category signal is useful as a weak prior rather than as a dominant score.
4. **Hard category filtering is model-dependent and especially risky for embeddings.** It is slightly positive for BM25+, nearly neutral to slightly negative for TF-IDF, and clearly harmful for embeddings, where Recall drops from 0.965746 to 0.905954.
5. **Category-filtered + rerank is never the best option.** It improves on the baseline, but it always remains below reranking the unfiltered candidate set with a soft category bonus.

11.2 Cross-encoder reranker diagnostics

The supplementary notebook `reranker-evaluation.ipynb` gives a useful closer look at the reranker. On category-filtered embedding candidates, it improves the combined score from 0.568914 to 0.608709, which again shows that reranking helps the ordering near the top of the list. At the same time, the notebook also shows concrete failure cases.

1. **Its main benefit is rank repair, not candidate discovery.** MRR rises sharply while Recall usually stays fixed, which is exactly the signature of a reranker working on an already-strong candidate set.
2. **Average gains do not prevent per-query failures.** The reranker notebook includes a query for which the first relevant document is pushed down from rank 1 to rank 30, even though the reranked system is better on average.
3. **The classifier is safest as a soft signal.** The reranker diagnostics and the full Phase 2 table agree that a weak category bonus is helpful, while aggressive hard filtering can remove relevant material before the reranker ever sees it.

For these reasons, we describe the cross-encoder as a practical second-stage repair model. The main takeaway is not “use one fixed value forever,” but rather “use reranking to improve ordering, and use the classifier as a soft signal instead of an aggressive filter.”

12 Final Submission Result

Source	Evidence	Interpretation
Saved offline evaluation run	0.635325	Best measured Stage 2 combined score from <code>retrieval-evaluation.ipynb</code> ; it comes from the embedding retriever with reranking and a soft category bonus.
Current submission notebook	<code>final_model=embedding,</code> <code>top_k=12500, top_m=45,</code> <code>bonus = 0.5, tags en-</code> <code>abled</code>	Describes what <code>kaggle-submission.ipynb</code> now exports. In the saved run, the notebook predicts 141 test categories, reranks 141 queries, and writes <code>solutions_SeaFour.csv</code> .
External public Kaggle result	0.62419	Hidden-test leaderboard result. It is a useful external check, but it should not be read as an exact replay of one local run.

Table 12: Offline metrics, the submission notebook run, and the public Kaggle score are reported separately.

The two scores 0.635325 and 0.62419 are reasonably close, which is encouraging. The offline comparison pointed us toward a strong Stage 2 direction, and the public leaderboard result supports that direction from the hidden test side. At the same time, the report does *not* claim that the offline tables, the submission notebook, and the Kaggle score all came from one identical run.

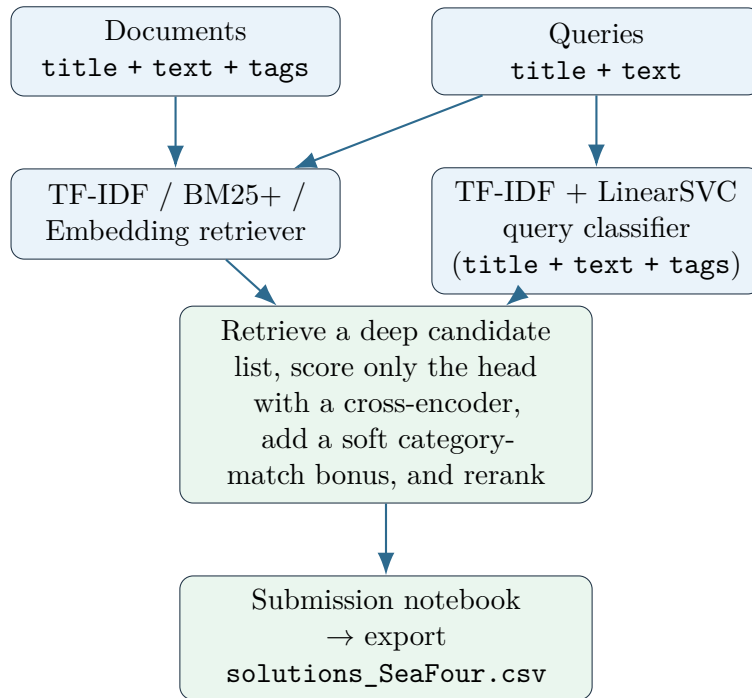


Figure 4: Final pipeline used to produce the submitted CSV.

13 Reproducibility

The notebooks include several choices that make reruns easier:

- cached document embeddings;
- cached TF-IDF and BM25 artifacts;
- explicit runtime path detection for local, Colab, and Kaggle environments;

- explicit random seeds and reranker training limits;
- schema-aware qrels parsing that matches the competition files;
- stable category-preserving sampling for visualization and downsampling;
- notebook-level documentation through section markdown and structured outputs.

A lightweight cross-encoder reranker is enabled in the current code defaults, but in this report we still separate the saved offline evaluation run from the current submission setup. The reranker notebook explains the effect of the main Stage 2 choices, while the public Kaggle score remains an external check.

14 AI Usage Declaration

Large language model assistance was used during the preparation of this revised notebook/report package. In particular, LLM tools were used to:

- help reorganize and clean parts of the notebook and project documentation;
- suggest debugging fixes and consistency checks for the local/Kaggle runtime setup;
- help revise and polish sections of the written report.

The final methodological choices, interpretations, and submission artifacts remain the responsibility of the team.

15 Future Improvements

The most important next step is better reranker development. The current cross-encoder already improves the top of the ranking, but its setup is still closer to a practical competition pipeline than to a deeply optimized second-stage ranking system. Future work should focus on more systematic tuning of rerank depth, candidate-set size, score calibration, and how cross-encoder scores are combined with first-stage retrieval signals. A stronger reranker model could also be explored, but the bigger goal is to make Stage 2 more deliberate instead of simply deeper.

This is also the path toward a genuinely strong information retrieval system. In a real deployment, the goal would not be to retrieve many documents and stop there. After broad candidate generation and reranking, the system should narrow the final output to a smaller and higher-quality set using explicit logic such as top- n cutoffs, score thresholds, confidence-based filtering, deduplication, category-aware constraints, or task-specific ranking rules. In that sense, the current pipeline shows a strong competition direction, but it should not yet be described as a complete production retrieval architecture.

16 Conclusion

We do not claim that returning 12500 documents would make sense in a real application. We used that depth here because the competition rewards broad coverage, not because it would be a realistic final user output. In practice, candidate generation can be wide, but the final returned set should be narrowed after reranking with explicit decision logic such as top- n cutoffs, score thresholds, confidence rules, or application-specific ranking criteria. Since careful reranker tuning is already a difficult task on its own, we leave the current system in a competition-oriented state instead of presenting it as production-ready.

The notebook and this report align with the full brief:

- they present the dataset and preprocessing explicitly;
- they compare TF-IDF, BM25+, and embeddings directly;

- they evaluate embeddings structurally and visually;
- they add category classification with document/query comparison;
- they use the classifier to improve retrieval in a way that is measurable and easy to justify;
- they preserve a reproducible path to Kaggle submission.

Empirically, the strongest **saved offline** Stage 2 variant in the project is the embedding retriever with reranking and a soft category bonus, which reached **0.635325** in `retrieval-evaluation.ipynb`. The **submission notebook run** reports `final_model=embedding`, `submit_top_k=12500`, `rerank_top_m=45`, `category_bonus=0.5`, and writes `solutions_SeaFour.csv`. The **public Kaggle score** of **0.62419** remains an external result. The main conclusion is not that every tuned value has to be copied exactly, but that the project learned the right interaction between its components: semantic retrieval gives strong candidates, the cross-encoder improves their order near the top, and the category classifier helps most when it is used as a soft signal rather than a hard filter. That is the clearest final story of the project.